AWS SERIES · MODULE 3 OF 8

Amazon S3: Storing Files for Your Apps and AI

A beginner's guide to Amazon S3: what an object store is, how buckets and keys work, how to keep data safe and cheap, and how S3 feeds documents into a GenAI (RAG) application — with hands-on steps in the console and on the command line.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: Amazon S3 · Storage classes · Lifecycle · Versioning · Bucket policies · Pre-signed URLs · Event notifications · VPC gateway endpoint · S3 Vectors
Before this module: Module 1 (Networking) and Module 2 (IAM). Sample: AcmeCorp HR's policy documents for an HR assistant

1. What S3 is
2. Buckets, objects and keys
3. What is inside an object
4. How you talk to S3
5. How safe your data is
6. Upload and download
7. Organising data with prefixes
8. Storage classes
9. Lifecycle rules
10. Versioning
11. Who can access a bucket
12. Encryption
13. Pre-signed URLs
14. Reaching S3 privately from a VPC
15. Event notifications
16. S3 in a GenAI (RAG) app
17. What S3 costs
18. What you can skip for now
19. Troubleshooting
20. Build it yourself: what you will build
21. Before you start
22. Build 1 · Create a bucket
23. Build 2 · Upload, list, download
24. Build 3 · Versioning: overwrite, delete, recover
25. Build 4 · Lifecycle rule
26. Build 5 · Lock it down
27. Build 6 · Share with a pre-signed URL
28. Build 7 · Send upload events to SQS
29. Build 8 · Clean up (same day)
30. Key takeaways

1 What S3 is



Figure 1. Three kinds of storage. S3 is object storage: whole files in, whole files out, over HTTPS.

Amazon S3 (Simple Storage Service) stores files for you in AWS. You do not attach it like a disk. Your app **sends** a whole file to S3 and **asks** for a whole file back, using simple web requests.

	S3 is good for	S3 is not
Data	Documents, PDFs, images, videos, logs, CSV/Parquet, backups, model files	A database — you cannot query rows inside a file (use DynamoDB/RDS)
Size	From 0 bytes up to **50 TB** per object; no limit on total	A disk — you cannot change bytes in the middle of a file; you replace the whole file
Access	Any app, Lambda, container or person with permission	A folder tree — "folders" are only part of the name (section 2)

> For GenAI work, S3 is where your **documents, datasets, embeddings exports, evaluation results and model artefacts** live.

2 Buckets, objects and keys

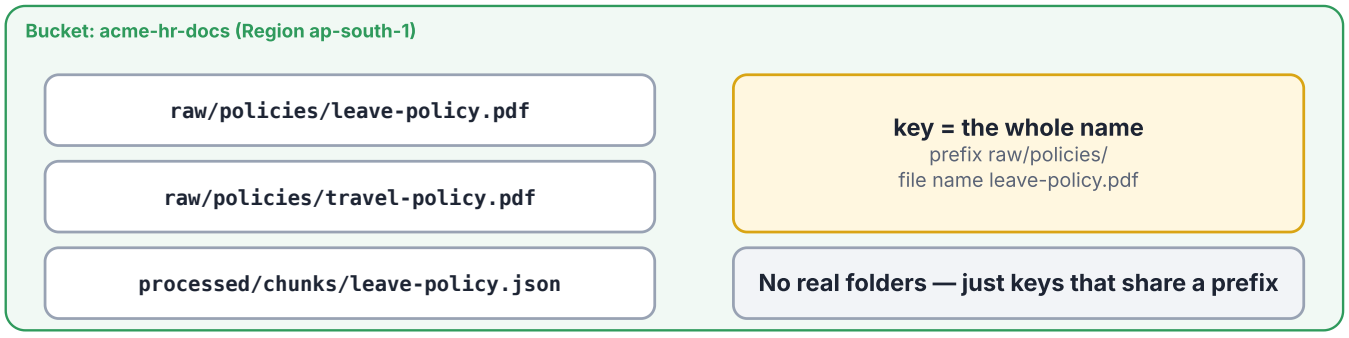


Figure 2. A bucket holds objects. Each object has a key (its full name). The "folders" are just the start of the key.

Word	Meaning	Example
Bucket	A container for objects. Lives in **one Region**. Its name is unique across **all AWS accounts**	`acme-hr-docs`
Object	One stored file plus information about it	a PDF
Key	The object's full name inside the bucket	`raw/policies/leave-policy.pdf`
Prefix	The start of a key. The console shows prefixes as folders	`raw/policies/`

Bucket name rules

- 3 to 63 characters: lowercase letters, numbers, hyphens (and dots, but avoid them).
- Must be unique worldwide. `docs` is taken; `acme-hr-docs-111122223333` (with your account ID) is safe.
- The name cannot be changed later. To "rename", create a new bucket and copy.

> Because "folders" are only part of the key, **renaming a folder** means copying every object to a new key and deleting the old ones. Choose your prefix layout early (section 7).

3 What is inside an object

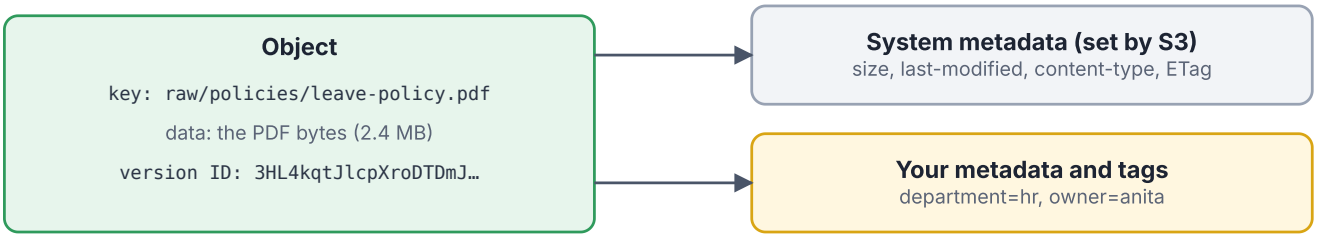


Figure 3. One object = key + data + metadata (+ a version ID if versioning is on).

Part	What it is	Can you change it later?
Key	The name	No — copy to a new key instead
Data	The file's bytes	Only by uploading a whole new version
System metadata	Size, date, content type (e.g. `application/pdf`), ETag (a checksum)	Content type: yes, on upload/copy
User metadata	Your own `x-amz-meta-…` values	Only by copying the object
Tags	Up to 10 key=value labels	Yes, any time
Version ID	Unique ID for each version (section 10)	No — set by S3

> Use **tags** (e.g. `department=hr`) for things that change or that you want to filter, bill or secure by. Tags can be used in lifecycle rules and IAM policies.

4 How you talk to S3

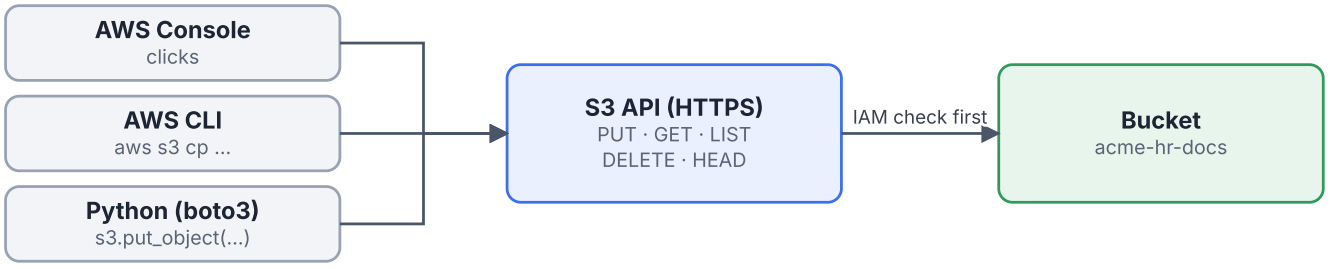


Figure 4. Every S3 action is an HTTPS request to the S3 API. The CLI, SDKs and console all make these calls for you.

Action	API call	CLI example
Upload	`PutObject`	`aws s3 cp file.pdf s3://acme-hr-docs/raw/`
Download	`GetObject`	`aws s3 cp s3://acme-hr-docs/raw/file.pdf .`
List	`ListObjectsV2`	`aws s3 ls s3://acme-hr-docs/raw/`
Delete	`DeleteObject`	`aws s3 rm s3://acme-hr-docs/raw/file.pdf`

Action	API call	CLI example
Read info only	`HeadObject`	`aws s3api head-object --bucket … --key …`

Address form	Example	Used by
S3 URI	`s3://acme-hr-docs/raw/policies/leave-policy.pdf`	CLI, Bedrock, Spark, most tools
HTTPS URL	`https://acme-hr-docs.s3.ap-south-1.amazonaws.com/raw/policies/leave-policy.pdf`	Browsers, pre-signed links
ARN	`arn:aws:s3:::acme-hr-docs/raw/*`	IAM and bucket policies

> Two CLI families: `aws s3 …` (simple, like file commands: `cp`, `ls`, `sync`, `rm`) and `aws s3api …` (one command per API call, every option). Start with `aws s3`.

5 How safe your data is

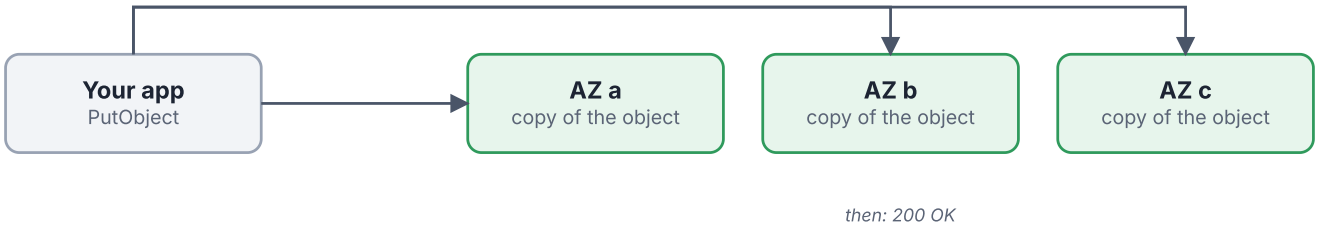


Figure 5. A successful upload is stored in at least three AZs before S3 says OK.

Promise	What it means for you
Durability: 99.999999999% ("11 nines")	S3 practically never loses an object. Data is lost by **people or scripts deleting it** — see versioning
Copies in 3+ AZs	An AZ can fail and your data is still there (except the One Zone storage class)
Strong consistency	After a successful upload, the very next read or list sees it. No waiting
Checksums	S3 checks the data on upload and download, so corrupted bytes are caught

6 Upload and download

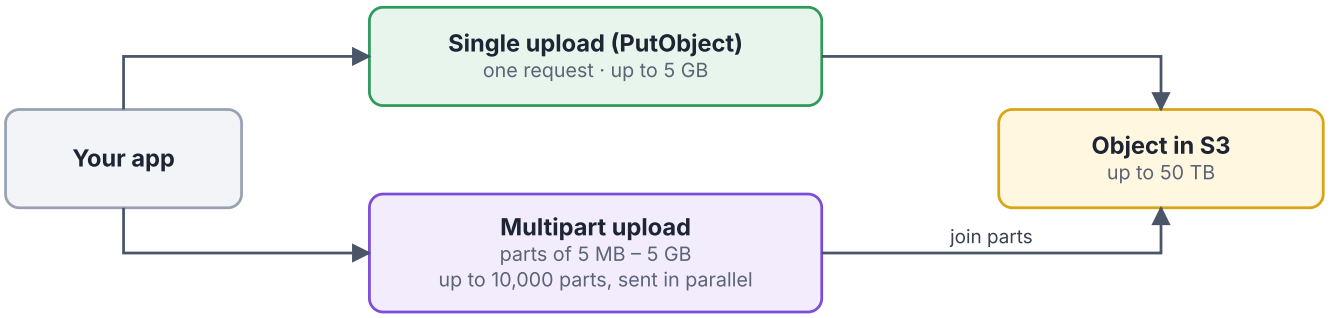
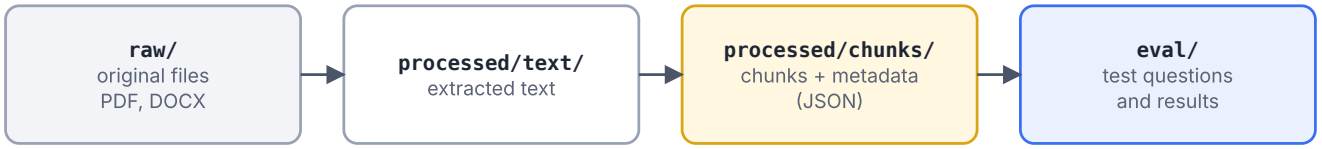


Figure 6. Small files go in one request. Big files go in parts, in parallel, then S3 joins them.

1. **Your app calls S3 with credentials** from an IAM role (on AWS) or your access keys (on your laptop).
2. **S3 checks permissions.** If not allowed: `403 AccessDenied`.
3. **Small file: one request.** Large file: the CLI and SDKs automatically switch to **multipart upload** (about 8 MB and up).
4. **S3 stores the copies** and answers `200 OK` with an ETag.
5. **To read, ask for bucket + key.** You can also ask for only part of the file (a byte range) — useful for large datasets.

```
aws s3 cp leave-policy.pdf s3://acme-hr-docs/raw/policies/          # upload one file
aws s3 cp s3://acme-hr-docs/raw/policies/leave-policy.pdf .        # download
aws s3 sync ./policies s3://acme-hr-docs/raw/policies/            # upload only new/changed files
aws s3 ls s3://acme-hr-docs/raw/ --recursive --human-readable      # list everything under raw/
```

7 Organising data with prefixes



Date parts in the key, e.g. raw/policies/2026/11/01/leave-policy.pdf, make "only today's files" easy to find

Figure 7. A simple layout for a RAG app: each stage writes to its own prefix.

Prefix	Holds	Who writes it
`raw/`	Files exactly as received. Never edited	Upload app, HR team
`processed/text/`	Text extracted from each file	Parsing job (Lambda)
`processed/chunks/`	Chunks ready for embedding	Chunking job
`eval/`	Test questions, answers, scores	Evaluation runs
`models/` or `artifacts/`	Prompt versions, fine-tuned model files	ML pipeline

- Keep **raw** data unchanged. If your chunking changes, you can rebuild everything from `raw/`.
- Rules (lifecycle, permissions, events) are set **by prefix**, so a clean layout makes them easy.
- Each prefix can handle thousands of requests per second (3,500 writes and 5,500 reads per second). Very busy apps spread data over several prefixes.

8 Storage classes

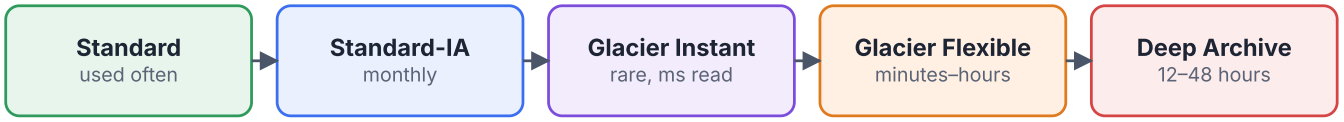


Figure 8. From hot to cold: storage gets cheaper, reading gets slower or costs more.

Each object has a **storage class**. It sets the price per GB and how fast (and at what cost) you can read it back.

Class	Use for	Read time	Watch out
S3 Standard	Data used often — app files, active RAG documents	Milliseconds	Default
S3 Intelligent-Tiering	You don't know how often it will be read	Milliseconds	Small monitoring fee per object; moves data for you
S3 Standard-IA	Read about once a month	Milliseconds	Fee per GB read; minimum 30 days
S3 One Zone-IA	Copies you can recreate	Milliseconds	Only one AZ
S3 Glacier Instant Retrieval	Archives you still open sometimes	Milliseconds	Minimum 90 days
S3 Glacier Flexible Retrieval	Archives	Minutes to hours	Must restore before reading
S3 Glacier Deep Archive	Keep for years (legal, audit)	Up to 12–48 hours	Cheapest; minimum 180 days

> Not sure? Use **Standard** for anything your app reads, and **Intelligent-Tiering** for large datasets with unclear use.

9 Lifecycle rules

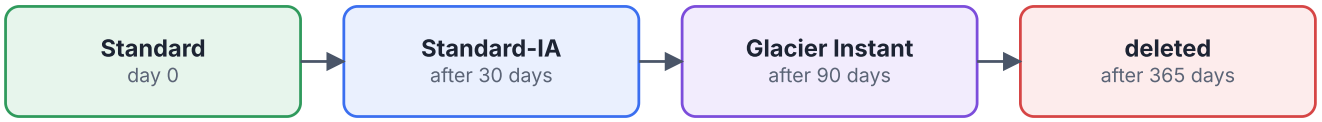


Figure 9. A lifecycle rule moves or deletes objects automatically as they age.

A **lifecycle rule** runs once a day. For objects matching a prefix or tag, it can **move** them to a cheaper class or **delete** them after N days.

Rule part	Example for AcmeCorp HR
Filter	Prefix `raw/`
Transition	Standard-IA after 30 days, Glacier Instant Retrieval after 90 days
Expiration	Delete after 365 days (only if policy allows)
Old versions	Delete noncurrent versions 30 days after they were replaced
Clean-up	Abort unfinished multipart uploads after 7 days

> Always add **abort unfinished multipart uploads**. Failed large uploads leave hidden parts that you keep paying for.

10 Versioning

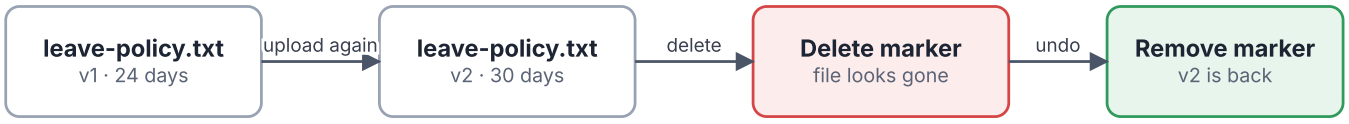


Figure 10. With versioning on, an overwrite adds a version and a delete only adds a marker.

With **versioning** on, S3 keeps every version of every object. Overwrites and deletes no longer destroy data.

You do	Without versioning	With versioning
Upload the same key again	Old file is gone forever	Old file kept as a **noncurrent version**
Delete	Gone forever	A **delete marker** hides it; the versions are still there
Undo	Impossible	Delete the marker, or download any old version by its ID

- Turn it on per bucket. After that it can be **suspended** but never fully switched off.
- Every version is billed as storage. Use a lifecycle rule to delete old versions after N days.
- Objects uploaded **before** versioning was turned on get the version ID `null`.

> Turn versioning on for any bucket that holds data you cannot easily recreate — like your `raw/` documents.

11 Who can access a bucket

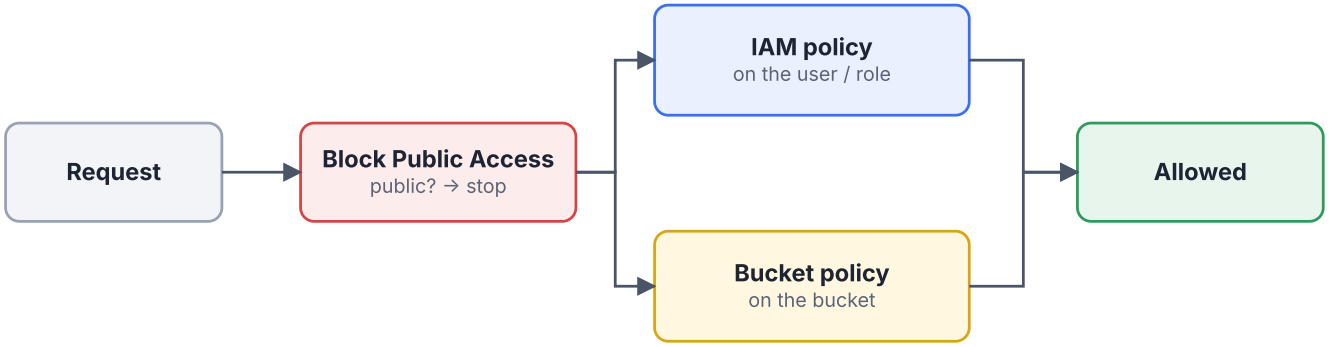


Figure 11. Every request passes these checks. Any explicit deny, or no allow at all, means 403.

A new bucket is **private**: only your account, and only users or roles whose IAM policy allows it, can use it.

Control	Set on	Use it for
IAM policy	A user or role	"This Lambda may read `acme-hr-docs/raw/*`" — the normal way inside one account
Bucket policy	The bucket	Rules for everyone: other accounts, AWS services, "HTTPS only", "only from our VPC"
Block Public Access	Bucket and account	A master switch that blocks any public access, whatever the policies say. **On by default**
ACLs	Objects	Old method. **Off by default** (Object Ownership = bucket owner enforced). Leave them off

```
// IAM policy for the HR assistant's Lambda role: read documents, write chunks
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::acme-hr-docs", "Condition": { "StringLike": { "s3:prefix": "raw/*" }
} },
    { "Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::acme-hr-docs/raw/*" },
    { "Effect": "Allow", "Action": "s3:PutObject", "Resource": "arn:aws:s3:::acme-hr-
docs/processed/*" } ] }
```

Common mistake: `s3:ListBucket` needs the **bucket** ARN (`…:::acme-hr-docs`); `s3:GetObject` needs the **object** ARN (`…:::acme-hr-docs/raw/*`). Give only one and you get `AccessDenied` .

12 Encryption

Type	Who holds the key	When to use
SSE-S3	AWS, fully managed	Default for every new object since 2023. Nothing to set up
SSE-KMS	A key in AWS KMS that you control	Need to audit who used the key, or limit who can decrypt (HR, Finance data)

Type	Who holds the key	When to use
In transit (HTTPS)	—	Always. Force it with a bucket policy (Build 5)

With SSE-KMS, a user needs **both** S3 permission **and** permission to use the KMS key. A missing KMS permission shows up as `AccessDenied` on download.

13 Pre-signed URLs

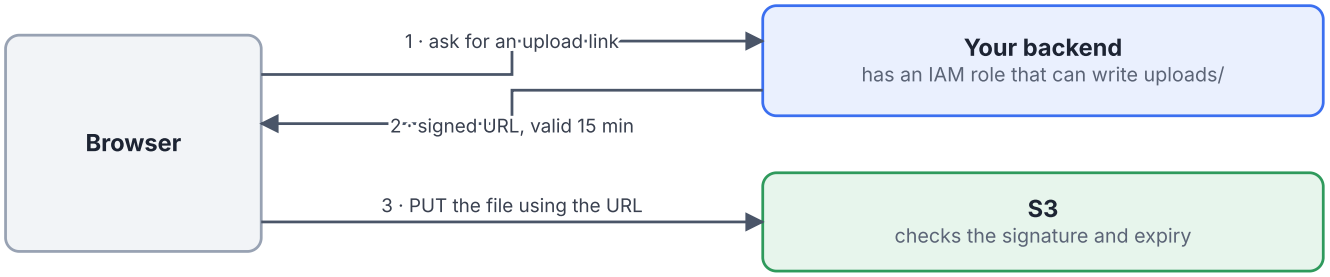


Figure 12. The backend signs a link. The browser uploads straight to S3; the file never passes through your server.

A **pre-signed URL** is a normal S3 link with a signature and an expiry time inside it. Anyone holding the link can do **that one action** on **that one object** until it expires — no AWS login needed.

```
import boto3
s3 = boto3.client("s3", region_name="ap-south-1")

# link to DOWNLOAD one file, valid 5 minutes
download_url = s3.generate_presigned_url(
    "get_object",
    Params={"Bucket": "acme-hr-docs", "Key": "raw/policies/leave-policy.pdf"},
    ExpiresIn=300)

# link to UPLOAD one file, valid 15 minutes
upload_url = s3.generate_presigned_url(
    "put_object",
    Params={"Bucket": "acme-hr-docs", "Key": "uploads/emp-1042/payslip.pdf"},
    ExpiresIn=900)
```

The link can only do what the **signer** is allowed to do. If the backend's role cannot write to `uploads/`, the link fails too.

14 Reaching S3 privately from a VPC

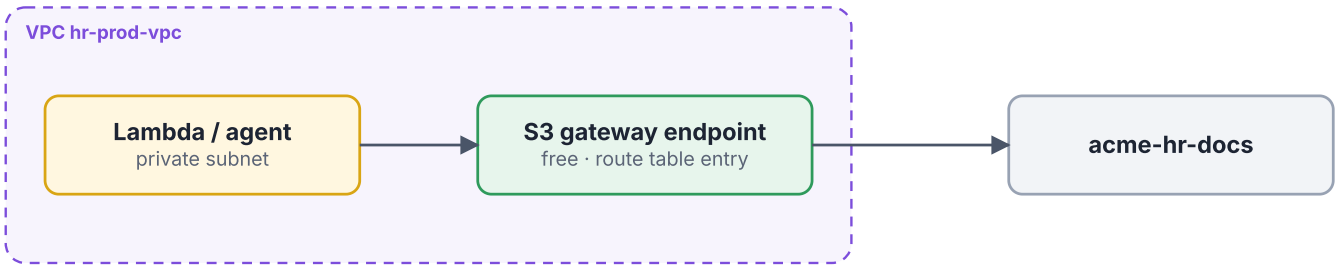


Figure 13. A gateway endpoint (Module 1) gives private subnets a free, private path to S3.

Apps in private subnets reach S3 through the NAT Gateway (costs per GB) or through a free **S3 gateway endpoint** (Module 1, section 16). For S3, always add the endpoint.

```
// Bucket policy: refuse any request that does not come through HR-prod's endpoint
{ "Effect": "Deny", "Principal": "*", "Action": "s3:*",
  "Resource": ["arn:aws:s3:::acme-hr-docs", "arn:aws:s3:::acme-hr-docs/*"],
  "Condition": { "StringNotEquals": { "aws:SourceVpce": "vpce-0hrprod123" } } }
```

Careful: this policy also blocks **you** in the console, because your browser is not inside the VPC. Add an exception for an admin role before applying it.

15 Event notifications

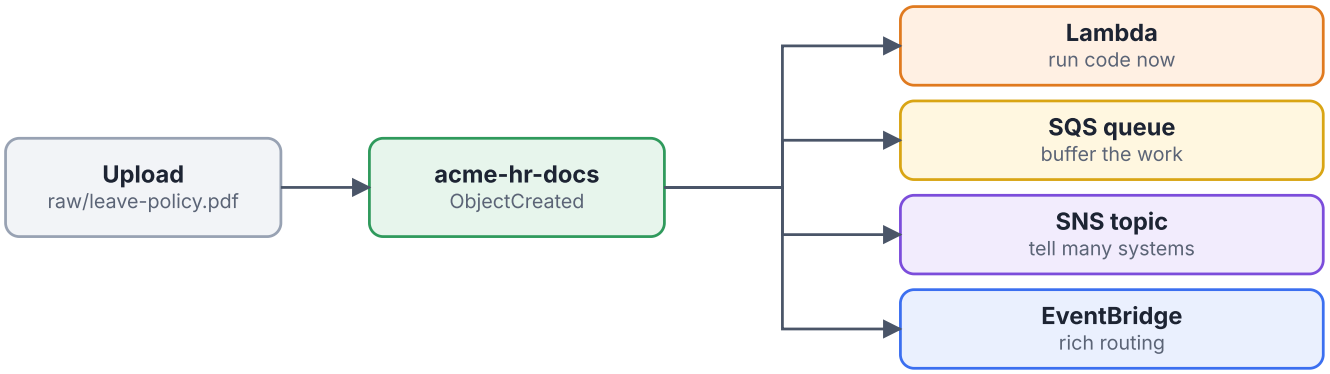


Figure 14. When an object is created or deleted, S3 can send a message to another service.

S3 can send an **event** when objects are created, deleted or restored. You filter by **prefix** and **suffix**, e.g. only `raw/` and only `.pdf`.

```
{ "Records": [ {
    "eventName": "ObjectCreated:Put",
    "eventTime": "2026-11-01T09:30:00.000Z",
    "s3": { "bucket": { "name": "acme-hr-docs" },
            "object": { "key": "raw/policies/leave-policy.pdf", "size": 2457600 } } } ] }
```

Destination	Use when
Lambda	Small, quick work per file (e.g. extract text)
SQS	Many files may arrive at once; workers take them at their own pace (Module 6)
SNS	Several systems must hear about the same file (Module 7)
EventBridge	Rich filtering or many targets (Module 8). Turn on once per bucket

Danger: a Lambda triggered by `raw/` that writes back into `raw/` triggers itself forever. Always write output to a **different prefix** (`processed/`) or bucket.

16 S3 in a GenAI (RAG) app

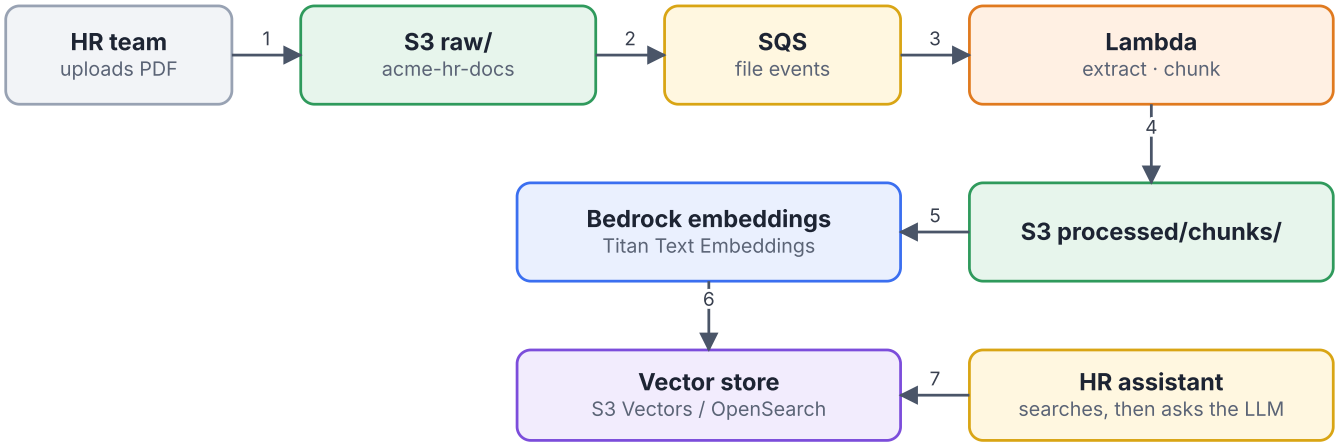


Figure 15. A RAG ingestion pipeline built on S3. The numbers match the steps below.

1. **HR uploads a policy PDF** to `raw/policies/`.
2. **S3 sends an** `ObjectCreated` **event** to an SQS queue.
3. **A Lambda takes the event,** downloads the PDF and extracts the text.
4. **It splits the text into chunks** and saves them to `processed/chunks/` (so you can re-embed later without re-parsing).
5. **Each chunk is turned into a vector** by an embeddings model on Amazon Bedrock.
6. **The vectors are stored** in a vector store — e.g. **Amazon S3 Vectors** (vector buckets inside S3) or OpenSearch.
7. **The HR assistant searches the vectors** for a question, then sends the best chunks to the LLM to write the answer.

> **Shortcut:** an **Amazon Bedrock Knowledge Base** can do steps 3–6 for you. You point it at the S3 prefix, press **Sync**, and it parses, chunks, embeds and stores the vectors (S3 Vectors is one of the storage options).

17 What S3 costs

You pay for	Notes
Storage	Per GB per month; cheaper in colder classes
Requests	Per 1,000 requests; writes (PUT, LIST) cost more than reads (GET)
Data out to the internet	Per GB downloaded from S3 to outside AWS
Retrieval	Per GB read from IA and Glacier classes
Free	Uploading data in; reading from S3 to services in the **same Region**; the S3 gateway endpoint

> Cost traps for AI projects: millions of tiny files (request charges), many old versions, unfinished multipart uploads, and reading S3 through a NAT Gateway instead of an endpoint.

18 What you can skip for now

Topic	One line
Replication (CRR / SRR)	Copy objects automatically to another bucket or Region
Object Lock	Make objects impossible to delete for a set time (legal / compliance)
Access Points	Separate entry points with their own policies for one big shared bucket
S3 Express One Zone	Fastest storage class, in special "directory buckets"
S3 Tables	Buckets built for Apache Iceberg tables (analytics)
Batch Operations, Storage Lens, Inventory	Tools for managing billions of objects

19 Troubleshooting

Error	Usual cause	Fix
`403 AccessDenied`	IAM or bucket policy does not allow it; ListBucket vs GetObject ARN mix-up; missing KMS permission	Check both policies and the ARNs (section 11)
`404 NoSuchKey`	Wrong key — keys are case-sensitive and include the full prefix	`aws s3 ls` the prefix and copy the exact key
`NoSuchBucket` / redirect	Wrong bucket name, or wrong Region	Check the name and use `--region ap-south-1`
`BucketAlreadyExists`	Someone in the world already has that name	Add your account ID to the name
`BucketNotEmpty` on delete	Objects **or old versions** still inside	Use **Empty** in the console (removes all versions)
Lambda times out reading S3	Private subnet with no NAT and no endpoint	Add the S3 gateway endpoint (Module 1)

20 Build it yourself: what you will build

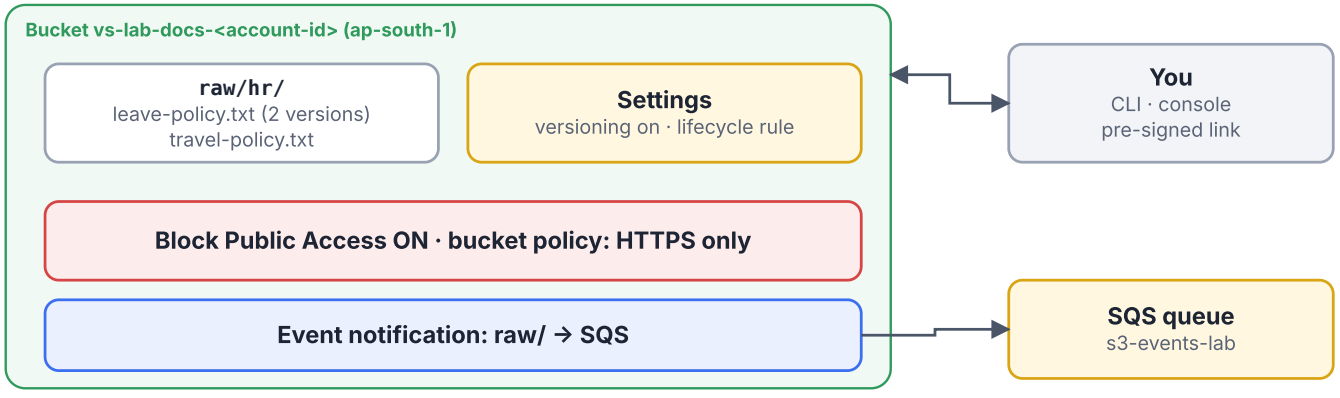


Figure 16. What you will build: one bucket with HR documents, protected, versioned, and sending events to a queue.

#	You do	Proves
1	Create a bucket	Names, Region, private by default (sections 2, 11)
2	Upload, list, download HR files	Keys and prefixes (sections 2, 4)
3	Turn on versioning; overwrite, delete, recover	Nothing is lost (section 10)
4	Add a lifecycle rule	Automatic cost control (section 9)
5	Check Block Public Access; force HTTPS	Security (section 11)
6	Share a file with a pre-signed link	Temporary access (section 13)
7	Send upload events to an SQS queue	Start of a RAG pipeline (sections 15, 16)
8	Clean up	No surprise bill

Cost: a few small files and one queue cost almost nothing. Still, do step 8 the same day.

21 Before you start

You need an AWS account and the AWS CLI set up — follow **Module 1, "Before you start"** if you have not done that yet. Open the **PyCharm Terminal** (Mac: zsh, Windows: PowerShell) and keep the **same window** open for all steps, because each step reuses variables like `$BUCKET`.

Mac

```
cd ~
mkdir -p ~/aws-s3-lab
cd ~/aws-s3-lab

ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
BUCKET="vs-lab-docs-$ACCOUNT"
echo $BUCKET
```

Windows (PowerShell)

```
cd ~
mkdir ~\aws-s3-lab
cd ~\aws-s3-lab

$ACCOUNT = aws sts get-caller-identity --query Account --output text
$BUCKET = "vs-lab-docs-$ACCOUNT"
echo $BUCKET
```

Adding your 12-digit account ID to the name makes it unique worldwide, so the create step will not fail with `BucketAlreadyExists`.

22 Build 1 · Create a bucket

Console

1. Search **S3** → **Create bucket**. Check the Region at the top right is **Asia Pacific (Mumbai)**.

2. **Bucket type:** General purpose. **Bucket name:** `vs-lab-docs-<your account id>`.
3. **Object Ownership:** ACLs disabled (default). **Block all public access:** ticked (default).
4. **Bucket Versioning:** Disable for now (Build 3 turns it on). **Encryption:** SSE-S3 (default).
5. **Create bucket**. Open it: it is empty, and the **Permissions** tab shows Block public access = On.

Command line — Mac

```
aws s3 mb "s3://$BUCKET" --region ap-south-1
aws s3 ls

# defaults you got for free:
aws s3api get-public-access-block --bucket $BUCKET
aws s3api get-bucket-encryption --bucket $BUCKET
```

Command line — Windows (PowerShell)

```
aws s3 mb "s3://$BUCKET" --region ap-south-1
aws s3 ls

# defaults you got for free:
aws s3api get-public-access-block --bucket $BUCKET
aws s3api get-bucket-encryption --bucket $BUCKET
```

23 Build 2 · Upload, list, download

Console

1. Open the bucket → **Create folder** → `raw` → open it → **Create folder** → `hr`.
2. Make two small text files on your computer: `leave-policy.txt` ("Leave policy: 24 days per year") and `travel-policy.txt` ("Travel policy: economy class for trips under 6 hours").
3. In `raw/hr/` → **Upload** → **Add files** → pick both → **Upload**.
4. Click `leave-policy.txt`: see its **Key**, **S3 URI**, **Object URL**, size, storage class, ETag.
5. Tick it → **Download**.

Command line — Mac

```
echo "Leave policy: 24 days per year" > leave-policy.txt
echo "Travel policy: economy class for trips under 6 hours" > travel-policy.txt

aws s3 cp leave-policy.txt "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3 cp travel-policy.txt "s3://$BUCKET/raw/hr/travel-policy.txt"

aws s3 ls "s3://$BUCKET/"                    # shows PRE raw/  (a prefix, not a folder)
aws s3 ls "s3://$BUCKET/" --recursive         # shows the full keys
aws s3api head-object --bucket $BUCKET --key raw/hr/leave-policy.txt

aws s3 cp "s3://$BUCKET/raw/hr/leave-policy.txt" downloaded.txt
cat downloaded.txt
```

Command line — Windows (PowerShell)

```
Set-Content -Path leave-policy.txt -Value "Leave policy: 24 days per year"
Set-Content -Path travel-policy.txt -Value "Travel policy: economy class for trips under 6 hours"

aws s3 cp leave-policy.txt "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3 cp travel-policy.txt "s3://$BUCKET/raw/hr/travel-policy.txt"

aws s3 ls "s3://$BUCKET/"                    # shows PRE raw/  (a prefix, not a folder)
aws s3 ls "s3://$BUCKET/" --recursive         # shows the full keys
aws s3api head-object --bucket $BUCKET --key raw/hr/leave-policy.txt

aws s3 cp "s3://$BUCKET/raw/hr/leave-policy.txt" downloaded.txt
Get-Content downloaded.txt
```

The console's **Create folder** just makes an empty object whose key ends in `/`. The CLI never needs it — uploading to `raw/hr/leave-policy.txt` creates the "folders" by itself.

24 Build 3 · Versioning: overwrite, delete, recover

Console

1. Bucket → **Properties** → **Bucket Versioning** → **Edit** → **Enable** → **Save changes**.
2. Edit `leave-policy.txt` on your computer to say "30 days per year" and upload it again to `raw/hr/`.
3. In `raw/hr/`, switch on **Show versions**: two versions — the old one has version ID `null` (uploaded before versioning).
4. Switch **Show versions** off → tick `leave-policy.txt` → **Delete** → type `delete`. The file disappears.
5. Switch **Show versions** on: you see a **Delete marker**. Tick only the marker → **Delete** → type `permanently delete`. The file is back.

Command line — Mac

```
aws s3api put-bucket-versioning --bucket $BUCKET --versioning-configuration Status=Enabled

echo "Leave policy: 30 days per year" > leave-policy.txt
aws s3 cp leave-policy.txt "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3api list-object-versions --bucket $BUCKET --prefix raw/hr/leave-policy.txt --query 'Versions[].
[VersionId,IsLatest,Size]' --output table

# read the OLD version (uploaded before versioning, so its ID is "null")
aws s3api get-object --bucket $BUCKET --key raw/hr/leave-policy.txt --version-id null old.txt
cat old.txt

# delete, see it is gone, then remove the delete marker to recover
aws s3 rm "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3 ls "s3://$BUCKET/raw/hr/"
MARKER=$(aws s3api list-object-versions --bucket $BUCKET --prefix raw/hr/leave-policy.txt --query
'DeleteMarkers[0].VersionId' --output text)
aws s3api delete-object --bucket $BUCKET --key raw/hr/leave-policy.txt --version-id $MARKER
aws s3 ls "s3://$BUCKET/raw/hr/"
```

Command line — Windows (PowerShell)

```
aws s3api put-bucket-versioning --bucket $BUCKET --versioning-configuration Status=Enabled

Set-Content -Path leave-policy.txt -Value "Leave policy: 30 days per year"
aws s3 cp leave-policy.txt "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3api list-object-versions --bucket $BUCKET --prefix raw/hr/leave-policy.txt --query 'Versions[].
[VersionId,IsLatest,Size]' --output table

# read the OLD version (uploaded before versioning, so its ID is "null")
aws s3api get-object --bucket $BUCKET --key raw/hr/leave-policy.txt --version-id null old.txt
Get-Content old.txt

# delete, see it is gone, then remove the delete marker to recover
aws s3 rm "s3://$BUCKET/raw/hr/leave-policy.txt"
aws s3 ls "s3://$BUCKET/raw/hr/"
$MARKER = aws s3api list-object-versions --bucket $BUCKET --prefix raw/hr/leave-policy.txt --query
'DeleteMarkers[0].VersionId' --output text
aws s3api delete-object --bucket $BUCKET --key raw/hr/leave-policy.txt --version-id $MARKER
aws s3 ls "s3://$BUCKET/raw/hr/"
```

25 Build 4 · Lifecycle rule

Console

1. Bucket → **Management** → **Create lifecycle rule** → name `raw-ageing`.
2. **Scope:** Limit the scope using filters → Prefix `raw/`.
3. Tick: **Transition current versions**, **Permanently delete noncurrent versions**, **Delete expired object delete markers or incomplete multipart uploads**.
4. Transitions: **Standard-IA** after 30 days, **Glacier Instant Retrieval** after 90 days. Noncurrent versions: delete after 30 days. Incomplete multipart uploads: 7 days.
5. Read the timeline summary at the bottom → **Create rule**.

Command line — Mac

```
cat > lifecycle.json <<'EOF'
{
  "Rules": [ {
    "ID": "raw-ageing",
    "Filter": { "Prefix": "raw/" },
    "Status": "Enabled",
    "Transitions": [ { "Days": 30, "StorageClass": "STANDARD_IA" },
                     { "Days": 90, "StorageClass": "GLACIER_IR" } ],
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  } ]
}
EOF

aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET --lifecycle-configuration
file://lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket $BUCKET
```

Command line — Windows (PowerShell)

```
@'
{
  "Rules": [ {
    "ID": "raw-ageing",
    "Filter": { "Prefix": "raw/" },
    "Status": "Enabled",
    "Transitions": [ { "Days": 30, "StorageClass": "STANDARD_IA" },
                     { "Days": 90, "StorageClass": "GLACIER_IR" } ],
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  } ]
}
'@ | Set-Content -Encoding ascii lifecycle.json

aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET --lifecycle-configuration
file://lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket $BUCKET
```

Nothing visible happens today — the rule acts on objects as they reach 30 and 90 days old. The point is that cost control is now automatic.

26 Build 5 · Lock it down

Console

1. Bucket → **Permissions** → confirm **Block public access (bucket settings): On** for all four settings.
2. **Bucket policy** → **Edit** → paste the JSON from the command-line block below (from `{` to `}`), replacing `$BUCKET` with your bucket name → **Save changes**.
3. Open `raw/hr/travel-policy.txt` → copy the **Object URL** → paste it in a browser: you get **AccessDenied**. The bucket is private, as it should be.

Command line — Mac

```
cat > https-only.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Sid": "HttpsOnly",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": [ "arn:aws:s3:::$BUCKET", "arn:aws:s3:::$BUCKET/*" ],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  } ]
}
EOF

aws s3api put-bucket-policy --bucket $BUCKET --policy file://https-only.json
aws s3api get-bucket-policy --bucket $BUCKET --query Policy --output text

# try to read the file without signing in: AccessDenied
curl "https://$BUCKET.s3.ap-south-1.amazonaws.com/raw/hr/travel-policy.txt"
```

Command line — Windows (PowerShell)

```
@"
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Sid": "HttpsOnly",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": [ "arn:aws:s3:::$BUCKET", "arn:aws:s3:::$BUCKET/*" ],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  } ]
}
"@ | Set-Content -Encoding ascii https-only.json

aws s3api put-bucket-policy --bucket $BUCKET --policy file://https-only.json
aws s3api get-bucket-policy --bucket $BUCKET --query Policy --output text

# try to read the file without signing in: AccessDenied
curl.exe "https://$BUCKET.s3.ap-south-1.amazonaws.com/raw/hr/travel-policy.txt"
```

Read the policy as: **deny everything** to **anyone** when the request is **not HTTPS**. Deny always wins, so even admins cannot use plain HTTP. On Windows we call `curl.exe`, because plain `curl` in PowerShell is a different command.

27 Build 6 · Share with a pre-signed URL

Console

1. Open `raw/hr/travel-policy.txt` → **Object actions** → **Share with a presigned URL**.
2. Set **5 minutes** → **Create presigned URL**. The link is copied.
3. Paste it in a browser (or send it to a friend): the file opens — no login needed.
4. Wait 5 minutes and open it again: **Request has expired**.

Command line — Mac

```
aws s3 presign "s3://$BUCKET/raw/hr/travel-policy.txt" --expires-in 300
# copy the printed link into a browser: it works for 5 minutes, then expires
```

Command line — Windows (PowerShell)

```
aws s3 presign "s3://$BUCKET/raw/hr/travel-policy.txt" --expires-in 300
# copy the printed link into a browser: it works for 5 minutes, then expires
```

Compare with Build 5: the plain Object URL was denied, the signed URL works. The signature carries **your** permission, for one object, for a limited time.

28 Build 7 · Send upload events to SQS

Console

1. Search **SQS** → **Create queue** → **Standard** → name `s3-events-lab`.
2. **Access policy** → **Advanced** → replace the JSON with a policy that allows `s3.amazonaws.com` to `sqs:SendMessage`, with conditions `aws:SourceArn = arn:aws:s3:::<your bucket>` and `aws:SourceAccount = <your account id>` (same as the command line) → **Create queue**.
3. Back in S3 → your bucket → **Properties** → **Event notifications** → **Create event notification**.
4. Name `raw-uploads`, **Prefix** `raw/`, tick **All object create events**, **Destination:** SQS queue → `s3-events-lab` → **Save changes**.
5. Upload any new file into `raw/hr/`. Then SQS → `s3-events-lab` → **Send and receive messages** → **Poll for messages** → open a message.

Command line — Mac

```
QURL=$(aws sqs create-queue --queue-name s3-events-lab --query QueueUrl --output text)
QARN=$(aws sqs get-queue-attributes --queue-url $QURL --attribute-names QueueArn --query
Attributes.QueueArn --output text)

# 1. let S3 (only from your bucket and account) send messages to the queue
cat > queue-attrs.json <<EOF
{"Policy": "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Effect\":\"Allow\",\"Principal\":
{\"Service\":\"s3.amazonaws.com\"},\"Action\":\"sqs:SendMessage\",\"Resource\":\"$QARN\",\"Condition\":
{\"ArnLike\":{\"aws:SourceArn\":\"arn:aws:s3:::$BUCKET\"},\"StringEquals\":
{\"aws:SourceAccount\":\"$ACCOUNT\"}}}]}"}
EOF
aws sqs set-queue-attributes --queue-url $QURL --attributes file://queue-attrs.json

# 2. tell the bucket to send ObjectCreated events under raw/ to the queue
cat > notification.json <<EOF
{ "QueueConfigurations": [ { "QueueArn": "$QARN", "Events": ["s3:ObjectCreated:*"],
    "Filter": { "Key": { "FilterRules": [ { "Name": "prefix", "Value": "raw/" } ] } } } ] }
EOF
aws s3api put-bucket-notification-configuration --bucket $BUCKET --notification-configuration
file://notification.json

# 3. upload a new policy and read the event
echo "Remote work policy: 2 days per week" > remote-policy.txt
aws s3 cp remote-policy.txt "s3://$BUCKET/raw/hr/remote-policy.txt"
aws sqs receive-message --queue-url $QURL --wait-time-seconds 10 --max-number-of-messages 5 --query
'Messages[].Body' --output text
```

Command line — Windows (PowerShell)

```
$QURL = aws sqs create-queue --queue-name s3-events-lab --query QueueUrl --output text
$QARN = aws sqs get-queue-attributes --queue-url $QURL --attribute-names QueueArn --query
Attributes.QueueArn --output text

# 1. let S3 (only from your bucket and account) send messages to the queue
@"
{"Policy": "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Effect\":\"Allow\",\"Principal\":
{\"Service\":\"s3.amazonaws.com\"},\"Action\":\"sqs:SendMessage\",\"Resource\":\"$QARN\",\"Condition\":
{\"ArnLike\":{\"aws:SourceArn\":\"arn:aws:s3:::$BUCKET\"},\"StringEquals\":
{\"aws:SourceAccount\":\"$ACCOUNT\"}}}]}"}
"@ | Set-Content -Encoding ascii queue-attrs.json
aws sqs set-queue-attributes --queue-url $QURL --attributes file://queue-attrs.json

# 2. tell the bucket to send ObjectCreated events under raw/ to the queue
@"
{ "QueueConfigurations": [ { "QueueArn": "$QARN", "Events": ["s3:ObjectCreated:*"],
    "Filter": { "Key": { "FilterRules": [ { "Name": "prefix", "Value": "raw/" } ] } } } ] }
"@ | Set-Content -Encoding ascii notification.json
aws s3api put-bucket-notification-configuration --bucket $BUCKET --notification-configuration
file://notification.json

# 3. upload a new policy and read the event
Set-Content -Path remote-policy.txt -Value "Remote work policy: 2 days per week"
aws s3 cp remote-policy.txt "s3://$BUCKET/raw/hr/remote-policy.txt"
aws sqs receive-message --queue-url $QURL --wait-time-seconds 10 --max-number-of-messages 5 --query
'Messages[].Body' --output text
```

The first message you see may be an `s3:TestEvent` — S3 sends it once to check it can reach the queue. The next one is your upload, with `eventName`, `bucket` and `key`: exactly what step 3 of the RAG pipeline (section 16) would read.

29 Build 8 · Clean up (same day)

Console

1. S3 → tick your bucket → **Empty** → type `permanently delete` → **Empty**. This removes **all versions** and delete markers.
2. Tick the bucket again → **Delete** → type the bucket name → **Delete bucket**.
3. SQS → tick `s3-events-lab` → **Delete** → type `confirm`.

Command line — Mac

```
# a versioned bucket must have every version and delete marker removed first
aws s3api list-object-versions --bucket $BUCKET --query '{Objects: [Versions, DeleteMarkers][].{Key:
Key, VersionId: VersionId}}' --output json > delete.json
aws s3api delete-objects --bucket $BUCKET --delete file://delete.json

aws s3 rb "s3://$BUCKET"
aws sqs delete-queue --queue-url $QURL

cd ~
```

Command line — Windows (PowerShell)

```
# a versioned bucket must have every version and delete marker removed first
aws s3api list-object-versions --bucket $BUCKET --query '{Objects: [Versions, DeleteMarkers][].{Key:
Key, VersionId: VersionId}}' --output json | Set-Content -Encoding ascii delete.json
aws s3api delete-objects --bucket $BUCKET --delete file://delete.json

aws s3 rb "s3://$BUCKET"
aws sqs delete-queue --queue-url $QURL

cd ~
```

> Why not just `aws s3 rb --force`? It deletes only the current files. With versioning on, old versions and delete markers stay, and the bucket refuses to be deleted (`BucketNotEmpty`).

30 Key takeaways

1. S3 stores whole files (objects) in buckets and is used over HTTPS — not a disk, not a database.
2. An object = key + data + metadata. "Folders" are just key prefixes; plan the layout early.
3. Data is kept in 3+ AZs with 11 nines durability. Data loss comes from deletes — so turn on versioning.
4. Pick a storage class by how often you read; lifecycle rules move and delete data automatically.
5. Buckets are private by default. Use IAM policies, a bucket policy, Block Public Access, and pre-signed URLs to share.
6. Private apps reach S3 through the free gateway endpoint.
7. S3 events start pipelines — like a RAG app's ingestion. Never write back to the prefix that triggers you.